Hallmarq Technical Documentation

Hallmarq records a fingerprint of AI-generated content on MST Blockchain so anyone can later check when it was registered and by whom. It needs one registry contract, one small SDK and one verifier page.

Goals and scope

Version 1 answers one question: was this exact content registered, by which account, and when? Everything else is deferred.

In scope

- One registry contract on MST testnet, with single and batched stamping
- A TypeScript SDK that normalizes, hashes, stamps and returns a receipt
- A static verifier page that checks content against the registry
- Published source on GitHub, verified on the explorer, with this document updated to match

Out of scope for v1

- Detecting whether content is AI-generated; Hallmarq only records claims
- Storing content, prompts or user data anywhere
- Image, audio and video matching that survives re-encoding (exact bytes only)
- Browser extension, platform plugins, token or fee logic of its own
- Any use of WASMify or SARAL until their interfaces are confirmed

Requirements

ID	Requirement	Kind	How it is checked
F1	A caller can register one content hash with a model label	Functional	Unit test: `stamp` then `verify` returns the record
F2	A caller can register up to 4,294,967,295 hashes with one Merkle root	Functional	Unit test: `stampRoot` then `verifyInBatch` returns true

ID	Requirement	Kind	How it is checked
F3	The first registration of a hash or root is permanent; a repeat reverts	Functional	Unit test: second `stamp` reverts with `AlreadyStamped`
F4	Anyone can verify without a wallet or an account	Functional	Verifier page works with no wallet installed
F5	The same content gives the same hash in the SDK and the verifier	Functional	Shared test vectors in both packages
N1	Content never leaves the user's device	Privacy	Code review; network log shows only RPC calls
N2	The contract has no owner, no upgrade path and no way to delete a record	Security	Source review; no `onlyOwner`, no proxy
N3	The model label is capped at 64 bytes	Cost	Unit test: 65 bytes reverts with `ModelTooLong`
N4	A verification result is available after finality (two blocks, about 6 seconds)	Performance	End-to-end test measures time to confirmed read
N5	Source is verified on the explorer so anyone can read what runs	Transparency	Explorer shows verified source and matching bytecode

MST platform facts

MST is a public EVM-compatible Layer 1, so Hallmarq can be plain Solidity deployed with standard tools. Every row below comes from MST's own whitepaper, website or grant page.

Topic	What MST states	Source
EVM	Fully EVM-compatible; Solidity, MetaMask, Remix, Hardhat and Truffle work with minimal or no change	Whitepaper, MST site

Topic	What MST states	Source
Consensus	Proof of Staked Authority (Parlia variant) with round-robin block production	Whitepaper
Block time and finality	About 3 seconds per block; finality after two consecutive justified blocks	Whitepaper
Native token	MSTC, used for transaction fees	Whitepaper
Fees	Site lists an average transaction fee of 0.001 MSTC	MST site
Throughput	2,234 TPS tested on testnet; 4,000+ TPS claimed for mainnet	Whitepaper
Explorer	Mainnet at mstscan.com; testnet at testnet.mstscan.com, built on Blockscout	Testnet explorer
SARAL	SDK and web protocol for sign-up with a mobile number or SSO (Google, Facebook, Twitter)	Protocols
WASMify	Runs Web2 application logic with WebAssembly and anchors the results on-chain	Protocols
Grants	initGrant up to $1,000; buildGrant up to $10,000; scaleGrant up to $50,000	Grants

Still to confirm

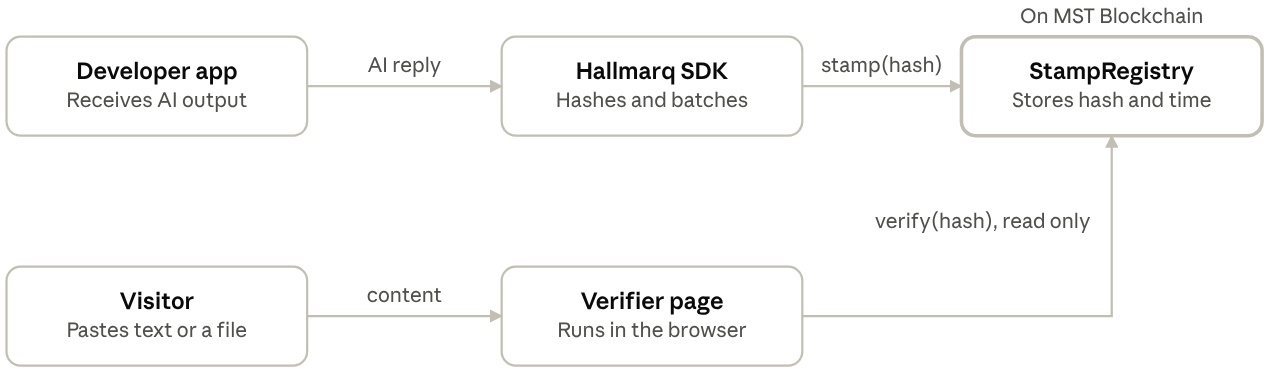
MST's developer docs site blocked automated access, so these items are not verified. Confirm each one before writing code or the grant application.

- [ ] Testnet chain ID, RPC URL and faucet (not found in public pages)
- [ ] Mainnet chain ID and RPC URL
- [ ] Exact gas limit and per-transaction cost for a contract write
- [ ] Solidity compiler version and EVM hardfork the chain supports
- [ ] Whether Blockscout contract verification is enabled on testnet
- [ ] WASMify interface: SDK, API or contract calls (only a high-level description is public)

- [ ] SARAL status and SDK: a 2025 article called it upcoming, the 2026 site describes it as available
- [ ] Current grant terms: the 2025 article said up to ₹1 Lakh; the grant page now lists dollar amounts

Architecture

Only a hash goes on-chain; the content stays with the app



StampAI architecture · 5 parts, 2 flows

The SDK sends only a hash to the registry. The verifier page reads the registry directly, so content never leaves the visitor's browser.

Hashing specification

The SDK and the verifier page must produce byte-identical hashes. Both follow these rules, and both are tested against the vectors below.

Text content

1. Decode the input as UTF-8. Reject input that is not valid UTF-8.
2. Replace every `\r\n` and every lone `\r` with `\n`.
3. Remove spaces and tabs at the end of every line.
4. Remove all whitespace at the very end of the text.
5. Encode the result as UTF-8 and compute keccak256. The 32-byte output is `contentHash`.

Leading whitespace and Unicode forms are left as they are. No Unicode normalization is applied, so "é" typed as one character and as two give different hashes.

Files

Hash the raw bytes with keccak256. No normalization of any kind.

Test vectors (computed with the keccak256 function used by Ethereum)

Input	After normalization	contentHash
`Hello, StampAI.`	unchanged	`0xd43d62a35d867840308edfb930a8716d939d78e3d38dff6a0b263a358a55a7c1`
`Hello, StampAI.` + two spaces + CRLF + `Second line.` + CRLF + CRLF	`Hello, StampAI.\nSecond line.`	`0x644f125e4c4fbfbcce78a3a427e043f1a4b1e2e1fd6b092bfaa325a67e206acd`

For comparison, the second input hashed without normalization gives `0x0816f49680ddc7160a0f40159cc485b50c57b9493f777f35b63c820d9f8505b9`. This is why normalization exists: copying text between systems often changes line endings and trailing spaces.

The zero hash (`0x00…00`) is reserved and rejected by the contract.

Contract specification

One contract, `StampRegistry`, stores one record per content hash. The first registration of a hash wins and can never be changed or removed. The sketch below is a design, not tested code.

Field	Type	Meaning
contentHash	bytes32	keccak256 of the normalized content (the mapping key)
registrant	address	Account that submitted the stamp

Field	Type	Meaning
model	string	Model label the registrant declares, for example "gpt-4o"
metaHash	bytes32	Optional hash of extra metadata kept off-chain; zero if unused
timestamp	uint64	Block timestamp when the stamp was mined

```
function stamp(bytes32 contentHash, string calldata model, bytes32 metaHash)
external;
function stampRoot(bytes32 root, uint32 count, string calldata model)
external;
function verify(bytes32 contentHash) external view returns (Stamp memory);
function verifyInBatch(bytes32 root, bytes32 contentHash, bytes32[] calldata
proof) external view returns (bool);
event Stamped(bytes32 indexed contentHash, address indexed registrant, string
model, uint64 timestamp);
```

Batching. `stampRoot` records the Merkle root of many hashes in one transaction, so cost stays flat as volume grows. The SDK keeps the proof for each leaf and the verifier page checks it with `verifyInBatch`.

Rules. A second `stamp` for the same hash reverts. The `model` string is capped at 64 bytes. No admin keys, no upgrade proxy and no way to delete a record.

Reference implementation

This source compiles without warnings with solc 0.8.26 and the optimizer at 200 runs; the optimized bytecode is about 3 KB. It has not yet been run against tests or deployed. A record whose `registrant` is the zero address means no record exists.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StampRegistry {
    struct Stamp { address registrant; uint64 timestamp; bytes32 metaHash;
string model; }
    struct Batch { address registrant; uint64 timestamp; uint32 count; string
model; }
```

```
    uint256 public constant MAX_MODEL_BYTES = 64;

    mapping(bytes32 => Stamp) private _stamps;
    mapping(bytes32 => Batch) private _batches;

    error EmptyHash();
    error ModelTooLong();
    error EmptyBatch();
    error AlreadyStamped(bytes32 key);

    event Stamped(bytes32 indexed contentHash, address indexed registrant,
string model, uint64 timestamp);
    event BatchStamped(bytes32 indexed root, address indexed registrant,
uint32 count, string model, uint64 timestamp);

    function stamp(bytes32 contentHash, string calldata model, bytes32
metaHash) external {
        if (contentHash == bytes32(0)) revert EmptyHash();
        if (bytes(model).length > MAX_MODEL_BYTES) revert ModelTooLong();
        if (_stamps[contentHash].registrant != address(0)) revert
AlreadyStamped(contentHash);
        uint64 ts = uint64(block.timestamp);
        _stamps[contentHash] = Stamp(msg.sender, ts, metaHash, model);
        emit Stamped(contentHash, msg.sender, model, ts);
    }

    function stampRoot(bytes32 root, uint32 count, string calldata model)
external {
        if (root == bytes32(0)) revert EmptyHash();
        if (count == 0) revert EmptyBatch();
        if (bytes(model).length > MAX_MODEL_BYTES) revert ModelTooLong();
        if (_batches[root].registrant != address(0)) revert
AlreadyStamped(root);
        uint64 ts = uint64(block.timestamp);
        _batches[root] = Batch(msg.sender, ts, count, model);
        emit BatchStamped(root, msg.sender, count, model, ts);
    }

    function verify(bytes32 contentHash) external view returns (Stamp memory)
{
        return _stamps[contentHash];
    }
```

```
    function getBatch(bytes32 root) external view returns (Batch memory) {
        return _batches[root];
    }

    function verifyInBatch(bytes32 root, bytes32 contentHash, bytes32[]
calldata proof) external view returns (bool) {
        if (_batches[root].registrant == address(0)) return false;
        bytes32 computed = keccak256(abi.encodePacked(bytes1(0x00),
contentHash));
        for (uint256 i = 0; i < proof.length; ++i) {
            bytes32 p = proof[i];
            computed = computed <= p
                ? keccak256(abi.encodePacked(bytes1(0x01), computed, p))
                : keccak256(abi.encodePacked(bytes1(0x01), p, computed));
        }
        return computed == root;
    }
}
```

Errors

Error	Raised when
`EmptyHash`	`contentHash` or `root` is all zeros
`ModelTooLong`	`model` is longer than 64 bytes
`EmptyBatch`	`count` is zero
`AlreadyStamped(key)`	The hash or root is already registered

Design notes. Batches and single stamps are separate namespaces. A hash stamped alone is not found by `verifyInBatch`, and the reverse. The contract does not check that `count` matches the tree; it is the registrant's claim, shown by the verifier as such.

Merkle batch specification

Batching lets one transaction cover many items. The tree is built off-chain by the SDK and checked on-chain by `verifyInBatch`.

1. **Leaf.** `leaf = keccak256(0x00 || contentHash)`

2. **Node.** `node = keccak256(0x01 || min(a, b) || max(a, b))`, comparing the two children as 256-bit numbers
3. **Odd node.** When a level has an odd number of nodes, the last one moves up unchanged
4. **Root.** The single node at the top; a batch of one has `root = leaf`
5. **Proof.** The list of sibling hashes from the leaf up to the root, in order

The `0x00` and `0x01` prefixes keep a leaf from ever being mistaken for an inner node, which blocks a known forgery on Merkle trees. Sorting each pair means the verifier needs no left/right flags.

Worked example with three items: the strings `A`, `B` and `C`, hashed directly as UTF-8.

Step	Value
contentHash of `A`	`0x03783fac2efed8fbc9ad443e592ee30e61d65f471140c10ca155e937b435b760`
contentHash of `B`	`0x1f675bff07515f5df96737194ea945c36c41e7b4fcef307b7cd4d0e602a69111`
contentHash of `C`	`0x017e667f4b8c174291d1543c466717566e206df1bfd6f30271055ddafdb18f72`
leaf of `A`	`0x55f349f6522044c9a77cb1f0c1e5c163b7905fa659abb4fed210ac8edefd6eb5`
leaf of `B`	`0x6e54f249b3a06dd774b95c8cc3b224c2af3e2abec031a82a0b83dddfa7459b1b`
leaf of `C`	`0x102b526fdb64add4fe15c0d20bc310b7d4aa6db14f3175484ec8775f40acf0b0`
node of leaves `A` and `B`	`0xa39369d497651a3868c1dc53d267b09a133ab055be78542f2f70e47d5be2d313`
root (node above, with leaf `C`)	`0xf0ea196d1197d65be1e15bf066760389c1301856f4de16b280ef6fc4f55275c0`

Proof for `A`: the leaf of `B`, then the leaf of `C`. Proof for `C`: the node of `A` and `B` alone. These values come from a script that follows the rules above; they have not yet been checked against the contract, and that check is a required test.

SDK

The SDK is a small TypeScript package that hashes content and talks to the registry through ethers.js. A developer should need three lines to stamp a response.

1. **Normalize.** Convert the content to UTF-8, trim trailing whitespace and normalize line endings to `\n`. Files are hashed as raw bytes.
2. **Hash.** Compute keccak256 of the normalized bytes. Only this hash leaves the app.
3. **Submit.** Call `stamp` for a single item, or queue hashes and call `stampRoot` on a timer or at a size limit.
4. **Return a receipt.** The caller gets the hash, transaction hash, block number and, for batches, the Merkle proof.

```
const receipt = await hallmarq.stamp(reply, { model: "gpt-4o" });
// { contentHash, txHash, blockNumber, proof? }
```

The SDK never sends prompts or content to MST or to any Hallmarq server. Signing uses the developer's own key.

SDK reference

This is the proposed public interface for v1. Names and defaults can change during the build; this section is updated when they do.

```
const hallmarq = new Hallmarq({
  rpcUrl,            // MST RPC endpoint (to be confirmed)
  chainId,           // MST chain ID (to be confirmed)
  registryAddress,   // deployed StampRegistry
  signer,            // an ethers Signer; the SDK never stores keys
  confirmations: 2,  // blocks to wait before resolving
  batch: { maxItems: 256, maxWaitMs: 30000 },
});
```

Method	Returns	What it does
`hashText(text)`	`0x` hex string	Normalizes and hashes text per the hashing specification
`hashBytes(bytes)`	`0x` hex string	Hashes raw bytes with no normalization

Method	Returns	What it does
`stamp(content, { model, meta? })`	`Promise<Receipt>`	Sends one `stamp` transaction and waits for confirmations
`stampBatched(content, { model })`	`Promise<Receipt>`	Adds the hash to the open batch for that model; resolves when the batch is confirmed
`flush()`	`Promise<void>`	Sends every open batch now
`verify(content)`	`Promise<VerifyResult>`	Looks the hash up on-chain; read only, needs no signer

Receipt has `contentHash`, `txHash`, `blockNumber`, `registrant`, `model` and, for batched stamps, `root` and `proof`. Developers must keep `proof`, because the chain stores only the root.

VerifyResult has `status` (`found`, `found-in-batch` or `not-found`) and, when found, the registrant, model label, block number and timestamp.

Batching behaviour

- One open batch per model label, because `stampRoot` records a single label per root
- A batch is sent when it reaches `maxItems` or has waited `maxWaitMs`, or when `flush()` is called
- A batch containing a hash that is already stamped is still valid; the duplicate is detected only by the single-stamp path
- The SDK retries a failed send up to 3 times with growing delays, and never resends once a transaction has been mined

Errors: `InvalidUtf8`, `ModelTooLong`, `AlreadyStamped`, `NotConfirmed` (the transaction was sent but confirmations did not arrive in time) and `NetworkError`.

Verifier page

A static web page that checks content against the registry in the visitor's browser. It needs no server and no wallet because it only reads from the chain.

- The visitor pastes text or drops a file; the page applies the same normalization and hash as the SDK.

- It calls `verify` on a public RPC. If a record exists it shows the registrant address, model label, block number and time, with a link to the transaction on the MST explorer.
- If none exists it shows "No record found" and says that this does not prove the content is human-made.
- For batched stamps the visitor also supplies the receipt's proof, and the page calls `verifyInBatch`.
- Content with any edit, even one character, hashes differently and shows no record.

Verification algorithm

The verifier page follows these steps, in order, entirely in the browser.

1. Read the input. Text is normalized; a dropped file is read as raw bytes.
2. Compute `contentHash` with keccak256.
3. Call `verify(contentHash)`. If `registrant` is not the zero address, show **Registered**.
4. Otherwise, if the visitor supplied a proof file (the SDK's receipt), read its `root` and `proof` and call `getBatch(root)` and `verifyInBatch(root, contentHash, proof)`. If both succeed, show **Registered in a batch**.
5. Otherwise show **No record found**.

Result	What the page shows	What it says to the visitor
Registered	Registrant address, model label, block number, UTC time, link to the explorer	"This exact content was registered by this account. The model label is declared by the registrant."
Registered in a batch	The same, plus the batch size and root	"This content is one of N items registered together. N is declared by the registrant."
No record found	The computed hash	"No record exists for this exact content. This does not mean it was written by a person."
Network error	The error and a retry button	"The registry could not be reached. This is not a verification result."

The page also shows the computed hash in every case, so a technical user can search for it on the explorer directly. A batched stamp cannot be found from the content alone, because the root is not derivable from one item; the visitor needs the receipt. A later version could add an off-chain index of proofs.

Security and limits

A stamp proves that someone registered a hash at a given time. It does not prove who made the content or which model really produced it.

Risk	Effect	Mitigation
False model label	Registrant can name any model	Show the label as "declared by registrant"; later, add signed attestations from AI providers
Front-running	Someone copies a hash from the mempool and registers it first	Accept it for v1 and document it; a commit-reveal step is a later option
Content edited after stamping	Hash no longer matches	Verifier states that matches are exact only
Missing stamp	Absence proves nothing	Verifier wording says so explicitly
Key loss or theft	Attacker can stamp as the registrant	SDK supports a dedicated low-value signing key
Bugs in the contract	Permanent wrong records	Keep it under 100 lines, test thoroughly, and verify the source on the explorer

Testing plan

Each requirement above maps to at least one test. Tests run locally on Hardhat first, then once against testnet.

Layer	Tests	Passes when
Contract unit	Stamp and read back; repeat stamp reverts; zero hash reverts; 64-byte label accepted, 65-byte label reverts; zero-count batch reverts; repeat root reverts; events carry the right fields	All assertions hold
Contract property	For random sets of 1 to 500 hashes, every honest proof verifies; changing one byte of a proof, a leaf or the root makes it fail; a proof for a leaf prefix-swapped as an inner node fails	1,000 random runs pass
Cross-check	The contract's `verifyInBatch` accepts the worked example's proofs for `A` and `C`	Returns true for both, false for a wrong proof
Hashing	SDK and verifier page both reproduce the two test vectors and the Merkle example byte for byte	Outputs equal the published values
SDK	Batching triggers on size and on time; no resend after a mined transaction; `NotConfirmed` when blocks do not arrive	Mocked-chain tests pass
End to end	Stamp an LLM reply on testnet, wait for two confirmations, paste it into the verifier, see **Registered**	Works from a clean browser with no wallet
Static analysis	Run Slither on the contract and fix or document every finding	No unexplained findings

The end-to-end test also records time from send to a confirmed read, which is the measure for requirement N4.

Repository layout and deployment

```
hallmarq/
  contracts/StampRegistry.sol
  test/                 Hardhat tests, including the vectors
  scripts/deploy.ts     deploys and prints the address
  packages/sdk/         the TypeScript SDK
  packages/verifier/    the static verifier page
  docs/                 this document, exported
  hardhat.config.ts
  README.md
```

Deployment steps

1. Confirm the testnet chain ID, RPC URL and faucet in MST's developer docs (open items above).
2. Add MST testnet to `hardhat.config.ts` as a custom network with those values, and set the compiler to a version the chain supports.
3. Create a fresh wallet used only for deployment and fund it from the faucet. Keep its key in an environment variable, never in the repository.
4. Run the tests, then run `scripts/deploy.ts` and record the contract address and deployment transaction.
5. Verify the source on testnet.mstscan.com. The explorer is built on Blockscout, which supports source verification; confirm the exact method and API endpoint.
6. Send one real stamp and read it back with the SDK and the verifier page.
7. Write the address, chain ID, explorer link and commit hash into the README and into this document.

The contract has no admin functions, so a bad deployment is fixed by deploying a new contract and updating the address in the SDK and the verifier page.

Cost model

No per-stamp cost is published here because none has been measured. The site's average transaction fee of 0.001 MSTC is a network-wide figure, not the price of a registry write. Cost per stamp is gas used times gas price, and it is measured in three steps.

1. Run the tests with a gas reporter to get gas used per function on a local chain.

2. On testnet, send one `stamp` and one `stampRoot`, and read `gasUsed` and `effectiveGasPrice` from each receipt.
3. Compute cost per item for a single stamp, and for batches of 1, 16, 256 and 1,024 items.

Case	Record	Formula
Single stamp	gas used, fee in MSTC	gasUsed × gasPrice
Batch of N	gas used, fee in MSTC	gasUsed × gasPrice
Cost per item in a batch	fee in MSTC	batch fee ÷ N
Break-even batch size	N	smallest N where batch fee ÷ N is below a single stamp's fee

A batch costs almost the same for any N because it writes one record, so cost per item falls roughly in proportion to N. The measured table replaces this one in the grant application.

Build plan

The scope fits initGrant, which MST lists as up to $1,000 with four deliverables: a working prototype, a GitHub repository, a testnet deployment and basic technical documentation.

Week	Work	initGrant deliverable
1	Resolve the open items above; write and test `StampRegistry` with Hardhat; deploy to testnet	Testnet deployment
2	Build the TypeScript SDK with single and batched stamping; add unit tests	Working prototype
3	Build the verifier page; run an end-to-end demo that stamps an LLM reply and verifies it	Working prototype

Week	Work	initGrant deliverable
3	Publish the repository with a README; finish this document with real addresses and tested code	GitHub repository; technical documentation

Out of scope for v1: a browser extension, platform integrations, signed model attestations, and any use of WASMify or SARAL. Both can be added once their interfaces are confirmed.

After v1

Each extension below is separate work, fits a later grant tier, and depends on something outside this project.

Extension	What it adds	Depends on
Provider attestations	An AI provider signs the hash of its own output, and the contract checks the signature, so the model label stops being self-declared	A provider willing to sign
Proof index	A small service that stores batch proofs, so a visitor can verify batched content without a receipt	Hosting and a privacy review
SARAL sign-in	Stamping through a mobile number or Google sign-in, with no wallet setup	SARAL's SDK and its current status, both to confirm
WASMify processing	Heavier off-chain work, such as checking attestations, with results anchored on-chain	WASMify's interface, which is only described at a high level in public
Near-match for images and audio	Finds re-encoded or resized copies using perceptual fingerprints	An off-chain index and a clear false-match policy
Browser extension	Shows a mark beside stamped content on a page	The proof index

Glossary

Term	Meaning
Hash	A fixed-size fingerprint of data; any change to the data gives a different hash
keccak256	The hash function Ethereum uses, producing 32 bytes
Registrant	The account that sent the stamp transaction
Stamp	One on-chain record of a content hash, with registrant, model label and time

Term	Meaning
Batch	Many hashes recorded together as one Merkle root
Merkle root and proof	The single hash that stands for a batch, and the sibling hashes that show one item belongs to it
Finality	The point after which a block cannot be reversed; on MST, two consecutive justified blocks
EVM	Ethereum Virtual Machine, the runtime that executes Solidity contracts
PoSA	Proof of Staked Authority, MST's consensus method
MSTC	MST's native coin, used to pay transaction fees

Sources

- MST Blockchain White Paper v3.0.0
- MST Blockchain home page
- MST protocols page
- MST grant tracks
- MST testnet explorer
- MST Roadmap 2026 announcement
- India's Youth and MST Blockchain (2025)